08: Polynomial Regression

Stat 230 - Spring 2026

> **Note**
>
> This is the tutorial version of the R activity. If you prefer to use .qmd and RStudio directly, you can use https://aluby.github.io/stat230/activities/08-polynomial

1 Loading data

To load the wildfires data set, run the following code chunk:

```
wildfires <- read.csv("https://aluby.github.io/stat230/data/wildfires.csv")
wildfires$Year <- wildfires$Year - 1960 # years since 1960
wildfires$Acres <- wildfires$Acres / 1e6 # millions of acres
```

2 Fitting a polynomial regression model

To fit a polynomial regression model we still use the `lm()` command, but we expand our formula to include polynomial terms. To include polynomial terms in a regression model, we need to use the `I()` function to indicate that we want to calculate a polynomial term. For example, to fit the quadratic model we have already discussed in class, we use the following code:

```
quadratic_lm <- lm(Acres ~ Year + I(Year^2), data = wildfires)
```

Once you have your fitted model, we can explore it like we have with simple linear regression models.

3 Exploring a cubic model

Let's fit a cubic model to the wildfires data set. The cubic model has the form

$$\mu\{y|x\} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3.$$

Use the `lm()` command to fit the cubic model where `Year` is used to predict `Acres`.

```
#| min-lines: 3
# Write your lm() command here
```

To plot the fitted cubic model, you can use the `gf_point()` and `gf_lm()` functions from the `ggformula` package. The following code will create a scatter plot of the data and add the fitted cubic regression line:

```
#| fig-alt: "Scatterplot with Years since 1960 on the x-axis and millions of acres bu
gf_point(Acres ~ Year, data = wildfires, xlab = "Years since 1960", ylab = "Millions
  gf_lm(formula = y ~ poly(x, 3), linewidth = .5)
```

> **Note**
>
> In the `gf_lm()` layer we use the `poly(x, 3)` function to specify that we want to fit a cubic polynomial. You can use `poly()` to fit polynomials of any degree by changing the second argument, and you can also use this function within the `lm()` function to fit polynomial regression models if you'd like.

1. Does the cubic model appear to be necessary? Use the `summary()` function to explore the fitted model and run a hypothesis test for the cubic term. What do you conclude?

2. What degrees of freedom did R use for the t-distribution used to calculate the p-value for the test of the cubic term?

3. Do you notice anything curious about the inferential results for the linear and quadratic terms?

The issue here is that the polynomial terms for year are highly correlated with each other (i.e., year, year^2, and year^3 are correlated). This can lead to numerical instability and make it difficult to interpret the coefficients. This is a situation called **multicollinearity**. We'll talk more about this later. One way to remedy this issue in polynomial regression is to use orthogonal polynomials, which are uncorrelated with each other.

4 An alternative way to fit polynomials

To fit polynomial model with uncorrelated polynomial terms use the `poly()` function. For example, to fit a cubic model using orthogonal polynomials, we can run the following code:

```
cubic_lm_ortho <- lm(Acres ~ poly(Year, 3), data = wildfires)
```

Use the `summary()` function to explore the fitted model. What do you notice about the inferential results for the linear, quadratic, and cubic terms? How does this compare to the previous cubic model we fit? How does it compare to the quadratic model?

```
#| min-lines: 3
# Write your lm() command here
```

The method of constructing the polynomial terms in our regression model does not change our predictions, but it can change the inferential results for the polynomial terms.

---

5 Function quick reference

The following table summarizes the functions we learned today:

Function	Purpose
`lm(formula, data)`	Fit a linear model. For polynomial regression the formula should include polynomial terms or use `poly()`.
`I()`	Used to create polynomial terms in a regression model
`poly(x, degree)`	Create orthogonal polynomial terms